Saudi GRC Platform

Frontend Requirements

Source: `frontend-requirements.md`

17 — Frontend Implementation Requirements

Spec-level breakdown of the platform's UI so a frontend team can rebuild or extend it. Anchored to the actual React + Vite + TypeScript codebase at `tss-grc-ui-v2/src/`.

Sister docs: - `/05-UI-Screens/` — per-screen list + purpose - `/08-RBAC/` — RBAC matrix (what each role sees) - `/03-Workflow-Catalogue/workflow-lifecycles.md` — state machines

---

1. Navigation menu

Sidebar (left in LTR, right in RTL). 11 groups, max 10 items each (ServiceNow IRM / OneTrust pattern; see `Sidebar.tsx`).

Group	Items
My Work	Dashboard, Lifecycle Overview, Intake Inbox, Approvals, Digital Signatures, Notification Outbox
Risk Management	Enterprise Risk, Risk Lifecycle, Scenario Analysis, Emerging Risks, Incident Response, Root Cause Analysis, Change Management
Compliance & Policy	Compliance, Framework Packs, Policy Templates, Regulatory Feed, Control Crosswalks, Evidence, Attestation Campaigns, Ethics & Compliance, ESG
Audit	Audit Workspace, Audit Lifecycle, Evidence Chain, NCA ECC Export
Privacy	PDPL, Privacy Lifecycle, ROPA, Data Lifecycle, Cyber → PDPL Bridge, Data Sharing, Processor Due Diligence
Cybersecurity	Cybersecurity, Cyber Lifecycle, Framework Controls, Asset Register
Business Continuity	Business Continuity, BCM Lifecycle, Dependency Map, DR Plans, Crisis Management
Third-Party Risk	Third-Party Risk, Vendor Portal, Supply Chain Risk, Supplier Resilience, Contract Clauses, Fourth-party Exposure, Third-party Cyber Risk, Resilience Tests
Saudi Regulators	Regulatory Authorities, HRSD (Labor / GOSI / Nitaqat), MOC (Tasattur / Consumer / CR)
Service Catalog & Knowledge	Services Center, CAPA, Board Pack, Knowledge Base, Communications, Scheduled Reports
Administration	Users & Access, Committees & Meetings, SSO / Nafath, Workflow Versions, Connectors, Notifications, Notification Channels, File Upload, Onboarding, Settings, About

Topbar: organisation switcher, search (Cmd+K palette), language toggle (■■■■■ ↔ English), theme toggle, user menu.

Mobile (≤ 768px): sidebar collapses to a hamburger drawer.

2. Page list

The 80+ React pages live under `src/pages/{slug}/`. Top-level routes configured in `App.tsx`. Categories:

- **Modules** (one per backend module): risk, bcm, scm, tprm, cyber, privacy, compliance, evidence, incident, signature, workflow, reporting, dashboard.
- **Detail pages** (drill-in): RiskDetailPage, VendorDetailPage, ControlDetailPage, PolicyDetailPage, FindingDetailPage, DpiaDetailPage, EngagementDetailPage.
- **Lifecycle pipelines** (visual workflow stage view per module): risk-lifecycle, bcm-lifecycle, audit-lifecycle, cyber-lifecycle, privacy-lifecycle.
- **Portals** (external-facing): DsrPortalPage, VendorPortalPage, SpeakUpPortalPage, CaseTrackerPage, TrustCenterPage.
- **Lifecycle / cross-cutting**: lifecycle-overview, intake-inbox, approvals, signatures, notification-outbox, dashboard.
- **Cross-module hubs**: board-pack, communications, regulatory-change, control-crosswalk, bcm-dependency.
- **Admin**: admin, committees, workflows, workflow-versions, sso, mfa, white-label, settings, onboarding.

3. Component library

Under `src/components/`:

Category	Components
Layout	Shell, PageHeader, Sidebar, Topbar, Breadcrumb, DetailHeader, DetailLayout, DetailRail, OverviewStrip, PublicShell
UI primitives	Card, Kpi, Badge, Button, Tabs, Input, Select, ColorField, RangeField, Progress, Modal, ConfirmDialog, EmptyState, Stepper, Timeline, HijriDate, FilterModal, FormModal, LivePulse, BulkActionsBar, RowActions, RecordPicker, UserPicker, SavedViewSelector, LinkedChangesPanel, FileUploadField, ExportMenu
Data	DataTable, DataTableV2 (paginated, sortable, virtualised), useDataTable
Filter	FilterBuilder + useFilterState + operators
Viz	HeatMap (5×5), BarChart3D, Donut, CoverageMatrix, TimeSeries, LineChart, BarChart, PieChart
Workflows	WorkflowDesigner, ProcessDetailModal, ProcessLifecycleBar, ChainCatalog, ActiveWorkflowsPanel, LiveSlaPulse, SlaCountdown
Lifecycle	LifecyclePipeline, ActivityFeedDrawer
Comms	CommsPanel, AcknowledgeModal, ComposeModal, NewCommunicationModal
TPRM	TprmConsolidatedOverview, TprmOnboardingPipeline, TprmReassessmentDashboard
Compliance	CcmDashboard, PolicyLifecycleStrip, PolicyVersionHistory, NitaqatPanel, MocPanel, MocProcessInitiation, HrsdProcessInitiation
Risk	AppetiteBreachFeed, NewRiskModal
Privacy	PdplBreachConsole
Audit	EngagementMilestonePanel

Category	Components
Authority	RegulatorPostureCard, EntityAnchorsPanel, AnchorChips
Search	CommandPalette (Cmd+K)
RTL	DirectionalArrow
Intake	PromotedBadge, IntakeFormRenderer, ChangeRiskPanel

4. Table column patterns

Every list page uses the same shape:

Column type	Pattern
Code / ID	className="mono", fixed-width font, click navigates to detail
Title	bilingual switch via lang === 'ar' ? titleAr : titleEn
Status / lifecycle	
Severity / risk	colour-coded per tier
Numeric / score	mono, right-aligned
Date	mono ISO; tooltip with Hijri via
Owner	name + role badge
Actions	right-aligned, menu or inline
Expand chevron	left, 28px wide, toggles row-detail panel

5. Form fields

Built via with `FieldDef[]`:

Kind	Use
text	one-line strings (titles, codes, names)
textarea	multi-line (descriptions, justifications, rationale) — rows configurable
number	with min/max validation
date	YYYY-MM-DD; Hijri preview when AR
select	controlled options array

Spans (12-col grid): `3 | 4 | 6 | 8 | 12`. Modal renders responsive grid. Required fields prefixed with `*`. Validation surfaces inline + toasts on submit.

6. Buttons & actions

Variant	Use
primary	Save, Create, Approve, Sign
secondary	Filter, Export, Refresh
ghost	Cancel, Archive, secondary actions
danger	Reject, Delete, Terminate, Mark FAIL

All buttons render an optional leading icon via the `Icon` prop (Lucide). Size: `sm` / `md` / `lg`.

State-aware visibility: each row's action set is computed from current `status` against allowed transitions (state machine).

7. Filters

Per-page filters via with definitions (text, select, multiselect, date-range). Saved views via . Column-header filters via `DataTableV2` for quick narrow.

Free-text search: page-level + global Cmd+K (`CommandPalette`).

8. Modals

Modal	Trigger	Purpose
	"New X" button	Create / edit record
	Destructive actions	Confirm Delete / Reject / Cancel
	"Filter" button	Build a filter
	Communications	Outbound message
	Acknowledgement	Subject ack
	Workflow row click	Show workflow detail + actions

All modals are esc-dismissable, focus-trap, render in a portal, backdrop-blur. RTL flips margin-inline-start/end.

9. Wizard steps

Multi-step flows use + own state. Wizards in the platform: - **TprmOnboardingPipeline** (TPRM Onboarding tab) - **OnboardingPage** (tenant onboarding) - **HrsdProcessInitiation** + **MocProcessInitiation** (intake flows) - **NewCommunicationModal** (compose multi-step)

Step shape: title, sub, status (PENDING / ACTIVE / DONE / SKIPPED). Stepper supports both vertical (rail) and horizontal layouts.

10. Dashboard widgets

Widget engine in `src/pages/dashboard/widgets.tsx`. Each widget has: - `id` (stable) - `titleKey` (translation key) - `defaultSize` (`sm` / `md` / `lg`) - `isKpi` flag - `category` (`kpi` / `chart` / `list` / `progress`) - `render(ctx)` function

Layout is per-user, persisted in localStorage via `useDashboardLayout`. Customise mode: drag to reorder, X to remove, click to add from catalogue. Reset-to-default supported.

Default layout: Saudi posture → KPI strip (intakes / overdue / CAB / critical risks) → **TPRM snapshot** (new) → ERM KPIs → Heatmap → Risks by severity → Frameworks → Activity feed.

11. Empty states

component used wherever a list is empty. Props: `Icon`, `title`, `sub`. Each module supplies its own copy through translation keys (e.g. `risksEmpty`, `riskEmptyHint`).

Pattern: empty state explains *why* (no data, filtered out, demo mode) and points to the next action ("Click New to add a record").

12. Validation messages

Two layers: 1. **Client-side** via FormModal's per-field rules (`required`, `min`, `max`, `maxLength`, regex). 2. **Server-side** via Bean Validation; errors surface as toasts with the field name. `ApiError` from `lib/api.ts` carries the structured body for inline display.

Validation messages translated via `messages*.properties` (server) and `translations/{en,ar}.ts` (client).

13. Arabic / English labels

Every user-visible string declared in two parallel files: `src/translations/en.ts` + `src/translations/ar.ts`. Keys are flat strings, mirrored 1:1. `useI18n()` hook returns: - `t(key)` — string lookup with optional `{var}` substitution - `lang` — current `'en' | 'ar'` - `setLang` — toggle

Saudi-native Arabic preference encoded in `memory/feedback_saudi_native_arabic.md` — prefer business Arabic over literal MSA translations.

14. RTL behaviour

Rule	Enforcement
Root direction	when lang === 'ar'
Logical CSS	margin-inline-start, padding-inline-end, border-inline-start — never left/right
Icons that flip	Wrapped in (e.g. ChevronRight becomes ChevronLeft in RTL)
Numeric formatting	ISO digits by default; Arabic-Indic supported via user pref
Date	Gregorian + optional Hijri overlay via
Tables	Header cells right-aligned in RTL; numeric columns stay right-aligned (universal convention)
Charts	x-axis flipped where meaningful; legend mirrored

15. Role-based UI visibility

`lib/auth.ts → hasAnyRole(...roles)` helper. Patterns:

- **Whole-page guard**: route guard at App.tsx level (e.g. `/admin` only for `PLATFORM_ADMIN` + `GRC_ADMIN`).
- **Button-level guard**: `hasAnyRole('DPO') && …`.
- **Tab-level guard**: tab definition list filtered by role.
- **Field-level guard**: PII fields hidden / masked unless user has `PRIVACY_READ`.

Demo mode: anonymous users get a read-only sample tenant.

16. Workflow status badges

with these tone tokens (defined in `styles/tokens.css`):

Tone	Use
primary	Default / informational
success	APPROVED / VERIFIED / IMPLEMENTED / ACTIVE

Tone	Use
warning	DRAFT / IN_REVIEW / PARTIAL / PENDING
danger	REJECTED / FAILED / EXPIRED / OVERDUE
info	IN_PROGRESS / MITIGATED
accent	Highlight / new

Each module has a `STATUS_TONE` map in its lib file (e.g. `SCENARIO_STATUS_TONE`, `DSA_STATUS_TONE`, etc.).

17. Evidence upload components

+ lib `fileUpload.ts`: - Accept-list configurable per evidence type (pdf, docx, xlsx, pptx, csv, json, png, jpg, zip). - Max size 50 MB; chunked upload above 10 MB. - On upload: SHA-256 computed client-side; sent + server re-verifies + writes to chain. - Per-tenant ACL: hidden upload-button for roles without `EVIDENCE_WRITE`.

18. Approval and digital-signature components

Component	Use
	Approve action
	Reject action
Inline reject reason prompt	window.prompt(t('xxxRejectReasonPrompt'))
	Confirm destructive action
Signature inbox card (SignaturesPage)	Pending signature transactions; Sign / Reject actions per row
Signature transaction modal	Shows the document hash + provider + signer; click "Sign" calls signTransaction() with browser-computed SHA-256

Signature flow: 1. Module fires `POST /api/signature/transactions` to request. 2. Signer's inbox shows REQUESTED row. 3. Signer reviews payload → clicks Sign → client computes SHA-256 → POST `/sign` with hash + cert reference. 4. Status SIGNED, audit logged.

19. Report export components

standardises every list page's export:

```
reportingApi.xxxPdf(lang)}
onXlsx={() => reportingApi.xxxXlsx()}
/>
```

Plus the Dashboard `Board Pack PDF / PPTX` buttons. All exports route through `lib/reporting.ts` which calls the backend `/api/v1/reporting/*` endpoints. Bilingual output: pass `lang` query param.

Audit log: every export writes an `audit.events` row with verb `export`.